PROJECT DOCUMENTATION / WORK COMPLETED

RETFound
From setup to evaluation.

A simple explanation of the local project: how we made an existing retinal-image model usable, tested its predictions, and created reports to inspect the results.

Prepared 25 October 2026 · Saved benchmark run: 23 October 2026 · Local project report

5	3	100	65%
DR severity grades	Datasets in the mixed test	Images in the mixed test	Exact grade accuracy in that test

What we achieved

An existing model became a usable local evaluation workflow.

We can supply a retinal photograph, obtain a predicted diabetic retinopathy (DR) grade, and compare the prediction with the dataset's reference label.

The starting point was the existing RETFound repository and an APTOS2019 fine-tuned model checkpoint. The local work added prediction scripts, Windows launchers, dataset preparation, evaluation scripts, and readable result reports.

> **Training status:** The evaluated model was already trained. The saved mixed-dataset evaluation explicitly records no new fine-tuning or threshold fitting on its samples. This work focused on running and evaluating that model.

Scope: reconstructed from the current repository, Git history, source code, and saved output files. The stages below explain the workflow; they are not a complete timestamped record of every earlier conversation or action. Upstream RETFound development is credited as the starting point, not local work.

The work, from the beginning

1 Start with RETFound and the saved model

We used the repository's retinal-image model implementation and an APTOS2019 checkpoint. A checkpoint is a file containing the model's learned weights. This checkpoint predicts five grades: 0 no DR, 1 mild, 2 moderate, 3 severe, and 4 proliferative DR.

Evidence: models_vit.py, README.md, and the checkpoint information in the saved results.

2 Make prediction convenient on Windows

We added a launcher that uses the existing `ml-homework` Conda environment. The prediction script accepts an image path, checks two possible checkpoint folder layouts, and selects CUDA, Apple MPS, or CPU when available. It also supports forcing CPU use.

Evidence: run_prediction.bat and predict_aptos.py. The saved mixed test ran on CUDA with PyTorch 2.5.1+cu121.

3 Build the single-image prediction process

Each image is converted to RGB, resized, cropped to 224 × 224 pixels, and normalized. The script loads the model weights with strict matching and checks that the model returns five finite scores. The highest-scoring grade becomes the prediction.

It displays all five softmax scores so we can inspect the model's output. These are model scores, not calibrated medical probabilities.

Evidence: predict_aptos.py. A no-image mode provides a technical smoke test using a zero-filled input.

4 Evaluate IDRiD and save visual evidence

A dedicated evaluation script compares predictions with IDRiD test labels and saves a summary, a predictions CSV, a confusion matrix, and six image-gallery pages. The saved full test contains 103 images, with 42 correct severity grades.

Evidence: evaluate_idrid.py and saved IDRiD summary. This is a separate evaluation from the 33-image IDRiD subset below.

5 Prepare a repeatable test across three datasets

The preparation script samples 34 APTOS2019, 33 IDRiD, and 33 MESSIDOR2 images from the RETFound-linked test archives. It uses the fixed random seed `20261023`, samples without replacement, and excludes hidden files and notebook copies.

It retains source information and image manifests so the selection can be traced. PAPILA is excluded because its labels concern glaucoma rather than DR.

Evidence: prepare_benchmark.py and source metadata in predictions.json.

6 Run the model and create an interactive report

The same APTOS-trained model is applied to all 100 images. The evaluator saves predicted grades, reference grades, all five scores, image hashes, and aggregate metrics. The saved run reports zero duplicate image hashes.

The HTML report includes image previews, dataset and grade filters, correct/incorrect filters, confusion matrices, and CSV export. Its thumbnails are embedded, so it works offline.

Evidence: evaluate_benchmark.py and the completed benchmark report.

7 Document the workflow and preserve the results

The README explains how to run prediction and the benchmark on Windows. Local Git commit `8c47a55` adds those instructions; the latest commit, `d2074ef`, adds prediction/evaluation scripts and the HTML report template. Both commits are dated 23 October 2026.

The template `benchmark_report.html` is filled by the evaluator; the ready-to-view results are in `results/benchmark100/index.html`.

What the saved results show

Two questions were measured: **exact grade** asks whether the model selected the correct severity from 0–4; **any DR** asks whether it correctly separated grade 0 from grades 1–4.

100-image mixed-dataset evaluation · 23 October 2026

Dataset	Images	Correct grades	Grade accuracy	Any-DR accuracy
APTOS2019	34	31 / 34	91.2%	97.1%
IDRiD	33	14 / 33	42.4%	78.8%
MESSIDOR2	33	20 / 33	60.6%	60.6%
Combined	100	65 / 100	65.0%	79.0%



Performance differed across datasets

The highest grade accuracy was on APTOS2019, the dataset used to fine-tune the checkpoint. IDRiD and MESSIDOR2 test how the same model transfers to other datasets. The observed differences show that the APTOS result does not describe performance on every dataset.

Binary accuracy hides important errors

Across all 100 images, the model missed DR in 11 images and flagged DR in 10 images labeled no DR. All 11 missed cases came from MESSIDOR2, where only 2 of 13 DR cases were detected (15.4% sensitivity).

> **Separate full IDRiD test:** 103 images; 42 correct and 61 incorrect grades; **40.78% grade accuracy**; macro F1 **0.3252**. These results should not be added to the mixed test as if they were a verified independent set of 103 additional images.

▶ **More detail about the mixed-test metrics**

> **Interpretation limits:** These are small-sample research results, not full benchmark scores or evidence of clinical readiness. Class proportions differ, and the saved metadata cannot independently rule out overlap with checkpoint training data. The tests measure the saved model and preprocessing used here.

What we now have

File or output	Purpose
predict_aptos.py	Predict the DR grade of a supplied image.
run_prediction.bat	Run single-image prediction using the Windows Conda environment.
evaluate_idrid.py	Evaluate IDRiD and generate CSV, summary, matrix, and galleries.
prepare_benchmark.py	Prepare the repeatable 100-image selection.
evaluate_benchmark.py	Run inference and calculate mixed-test metrics.
run_benchmark.bat	Run preparation and evaluation in sequence.
Interactive benchmark report	Inspect images, predictions, filters, scores, and confusion matrices.
JSON results · **CSV results**	Preserve machine-readable evidence for all 100 predictions.
IDRiD matrix · **First gallery**	Visual evidence from the separate IDRiD evaluation.
README.md	Setup context and run instructions.

Run the existing workflow

From PowerShell in the RETFound project folder, using the configured environment and dependencies:

```
# Predict the included sample image
.\run_prediction.bat
```

```
# Predict another retinal image
.\run_prediction.bat "C:\path\to\retinal-image.jpg"

# Prepare the sample and regenerate the benchmark results
.\run_benchmark.bat
```

The benchmark launcher needs network access when fetching missing images. Existing downloaded samples are reused. The older IDRiD script expects `checkpoints/APTOS2019/checkpoint-best.pth`; unlike the prediction and mixed-test scripts, it has no fallback for the nested checkpoint layout.

Proposed next steps · not completed

Use the findings to guide the next experiment.

1. **Inspect errors:** review missed MESSIDOR2 DR cases and under-graded severe/proliferative cases in the image reports.
2. **Broaden evaluation:** use larger held-out sets with adequate examples of every grade and verify separation from training data.
3. **Compare model choices:** evaluate dataset-specific checkpoints or a separately planned fine-tuning experiment under the same evaluation procedure.
4. **Improve reproducibility:** document the exact working dependencies and align the older IDRiD script with the current checkpoint folder layout.

Prepared from local evidence on 25 October 2026. This report summarizes existing saved runs; creating this page did not rerun inference or train a model. Main numerical sources: mixed-test JSON and IDRiD summary. The page itself needs no external fonts, libraries, or network connection; linked project files require the project folder.